

Computer Methods in Biomechanics and Biomedical Engineering: Imaging & Visualization

ISSN: 2168-1163 (Print) 2168-1171 (Online) Journal homepage: www.tandfonline.com/journals/tciv20

Human motion activity recognition and pattern analysis using compressed deep neural networks

Navita Kumari, Amulya Yadagani, Basudeba Behera, Vijay Bhaskar Semwal & Somya Mohanty

To cite this article: Navita Kumari, Amulya Yadagani, Basudeba Behera, Vijay Bhaskar Semwal & Somya Mohanty (2024) Human motion activity recognition and pattern analysis using compressed deep neural networks, Computer Methods in Biomechanics and Biomedical Engineering: Imaging & Visualization, 12:1, 2331052, DOI: [10.1080/21681163.2024.2331052](https://doi.org/10.1080/21681163.2024.2331052)

To link to this article: <https://doi.org/10.1080/21681163.2024.2331052>



© 2024 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.



Published online: 04 Apr 2024.



Submit your article to this journal [↗](#)



Article views: 3041



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 17 View citing articles [↗](#)

Human motion activity recognition and pattern analysis using compressed deep neural networks

Navita Kumari^a, Amulya Yadagani^b, Basudeba Behera^a, Vijay Bhaskar Semwal^b and Somya Mohanty^c

^aDepartment of Electronics and Communication Engineering, National Institute of Technology, Jamshedpur, India; ^bDepartment of Computer Science Engineering, University of North Carolina at Greensboro, Greensboro, NC, USA; ^cDepartment of Computer Science Engineering, Maulana Azad National Institute of Technology, Bhopal, M.P., India

ABSTRACT

This work presents an on-device machine learning model with the ability to identify different mobility gestures called human activity recognition (HAR), which includes running, walking, squatting, jumping, and others. The data is collected through an Arduino Nano 33 BLE Sense board with a sampling rate of 119 Hz, which is embedded with an Inertial Measurement Unit (IMU) sensor. The same board is used as a microcontroller to identify human gestures by developing an end-to-end edge computing application. A deep neural network model is trained and then compressed for deployment on the board to create a self-contained, embedded device capable of identifying the type of gesture performed. Three deep learning models, namely Multi-Layer Perceptron (MLP), Convolutional Neural Network – Long Short Term Memory (CNN-LSTM) & CNN-Gated Recurrent Unit (CNN-GRU), are evaluated in the identification of the mobility gestures. The observed accuracy of the models is 96%, 97.1% and 97.8%, respectively, MLP, CNN-LSTM & CNN-GRU across the different gesture categories. The study shows the utility of embedded devices with deep neural network-based models, which can provide low cost, minimal power usage, and meet data privacy requirements in HAR.

ARTICLE HISTORY

Received 28 February 2023
Accepted 11 March 2024

KEYWORDS

Deep learning; internet of Medical Things (IoMT); sensors; health monitoring; human activity recognition (HAR)

1. Introduction

1.1. Overview

Human walking activity recognition (HWAR) is an active research area in the field of biomedical engineering (Bijalwan et al. 2021), robotic walk generation (Jain et al. 2022), human posture (Zebin et al. 2015), push recovery stability analysis (Semwal and Nandi 2015) and diagnosis of many diseases like cognitive decline, freezing of walk, Parkinson, etc (Bartolo et al. 2020; Semwal et al. 2022a). HWAR refers to the identification and modelling of human mobility gestures, ie running, walking, jumping, etc, with the help of lower limbs. Recently, with the growth of the Internet of Things (IoT) and Nanodevices (Alharthi et al. 2019), HWAR has been identified as a key application domain for human health monitoring (Brull et al. 2020). With the advancements in such devices, it has become possible to perform computation on less powerful devices, such as microcontrollers/embedded systems. An advantage of such computational efficiency is the ability to support complex computation models developed from machine learning/deep learning algorithms (Banbury et al. 2000; Chen et al. 2021). Machine learning-based HWAR modelling has been utilised for video-based human surveillance (Brull et al. 2020), posture stability analysis (Semwal et al. 2018) and human health monitoring & bio-mechanical analysis (Semwal et al. 2022b).

1.2. Motivation

Specific areas of application for HWAR involve the measurement of various physical parameters of humans and using pattern discovery approaches to model activity. Within the domain, a key area of the research is gait rehabilitating and motion simulation. Outcomes of this area include the analysis of human posture stability and push recovery analysis which is useful in diagnosing the patient's neurological disorders, rehabilitation of patients and analysis of highly active (ie athletes) individuals (Hsu et al. 2017a). Prior research (Vrigkas et al. 2015) in the area traditionally utilised parametric models which are expensive in terms of computation cost & power consumption in real-time environment (De Capua et al. 2009). Furthermore, currently, existing devices, such as smart watches, rely on internet connectivity and cloud services to perform recognition of activity patterns using non-parametric machine learning models (Gravina et al. 2017). Transmission of patient data across the internet to cloud services leads to privacy issues (Behera et al. 2021). The experiment study of HWAR enables the human-robot interaction (HRI).

1.3. Recent advancement

The limitations of prior applications of HWAR have led to increased interest in the development of wearable devices that can perform activity recognition on patients (De Leonardis et al. 2018). Specifically, a microcontroller powered

by a single coin cell battery with only a few kilobytes of memory can provide continuous activity recognition. Available embedded hardware from companies, such as Arduino, Raspberry Pi, STM and nVIDIA have included Micro Electro-Mechanical System (MEMS) based Inertial Measurement Unit (IMU) sensors, such as accelerometer, gyroscope and magnetometer. In combination with low-powered CPUs such as ARM cortex M4 processors, such boards allow for the implementation of machine learning models on the device. Within this study, we explore the Arduino BLE nano sense board to develop an end-to-end approach to collect activity data, develop neural network-based models, explore model compression, and implement an inference model on the board for continuous activity recognition.

1.4. Author's contribution

Specifically, research work presents the development of human walking gesture recognition using deep learning models. Five different walking gestures namely, Jump, Run, Walk, Squat and others are evaluated within the study. To achieve the objective following tasks are performed:

- **Data Collection:** The gesture data is collected for five different gestures through the 9-degree IMU sensor using Arduino nano BLE sense-33. The data is captured at a sampling rate of 119 Hz, and sensed data is transmitted to a server using Bluetooth Low Energy (BLE).
- **Deep Learning Model Development:** The captured training data is used to develop and compare deep learning models based on – deep stacked multi-layer perceptron, convolution-based networks and sequential recurrent neural networks.
- **Performance Analysis & Optimisation of Model:** The identified models are then optimised for deployment on low-powered microcontrollers. We evaluate the compressed model performance based on pruning and quantisation approaches to develop a model that has 8× lower size and compute requirements than the original model which achieves similar or better accuracy than the larger models.
- **On-Device Deployment:** Finally, demonstrate the usability of the approach by loading the compressed model on the Arduino Nano BLE sense-33 board for the classification of gestures. The recognised gesture (along with the probability) by the board is transmitted via BLE and also identified by the multi-colour LED colour on the board.

This research paper presents all the aspects of data collection using an IMU sensor, classification of five different gestures using a deep learning model, and deployment to an edge device. A key advantage of such an approach is to perform on-device gesture recognition with low power utilisation while preserving the privacy of patient data (ie data is not transmitted to the cloud for recognition as with other smart devices). Furthermore, the approach can be used for the assessment of different walking activities and rehabilitation of individuals with abnormal gait.

1.5. Novelty of work

The innovation of work lies in designing of compressed deep learning model using pruning and quantisation to support edge computing and developing the compressed deep learning model by reducing the model using pruning and quantisation. It provides data accuracy, reduced size code with less computation and privacy of data (Das et al. 2022).

1.6. Organization of the paper

The whole paper is divided into five sections. The first section is an introduction that includes the author's contribution and background history. The next section is a literature review and analysis. The third section is a methodology, which covers the hardware and software specifications, algorithms and detailed flow work of human gesture recognition. The fourth section is the result and analysis section which covers the verification of the model. The final section is the conclusion.

2. Literature review

Interest in human activity recognition has increased significantly over the last few decades (Mim et al. 2023). In 2011, Studenski conducted a study in which he traced the gait velocity of over 34,000 seniors throughout 6 to 21 years (Studenski et al. 2011). Before Studenski's study, sex, age, smoking history, BMI, chronic conditions and blood pressure were considered to be good predictors of survival for a person. But in (Studenski et al. 2011) the authors show that along with the aforementioned predictors, gait speed, is a key indicator in the survival analysis of individuals. This finding boosted the interest of many researchers towards the effects of gait speed on the health of a person. This has led to an increase in the number of research findings on the significance of gait velocity as a prominent factor in the prediction of various health issues and chronic diseases including COPD, heart failure, stroke, kidney failure, etc.

Several studies have used various sensing modalities to capture gait data and analyse it. In 2014, Sigg used the Wi-Fi received Signal Strength Information (RSSI) on a smartphone to estimate the walking speed of that person (Sigg et al. 2016). In 2017, Chen-Yu Hsu, and Yuchen Liu built an algorithm to extract the gait speed from surrounding radio signals (Hsu et al. 2017b). They built a device almost the size of a regular painting to hang in the living room which will capture the radio signals and determine the walking speed of the person. But this is only useful for measuring the speed within the covered radio signal range of the device, ie inside the home. We cannot move it to outdoor places and use it. This has led to the development of on-body devices which can accurately monitor the gait movement of individuals. In 2019, Shi interpreted the changes in a person's walking speed and direction and presented a review of wearable ankle robots in post-stroke rehabilitation of gait (Shi et al. 2019). In 2019, Alharthi also presented a review on continuous monitoring of human gait using deep learning for health conditions (Alharthi et al. 2019).

Today gait velocity is measured by physical therapists or clinicians during medical visits. The person is asked to comfortably walk for a small distance (about 4 to 10 metres) and perform the various activities (Gupta et al. 2020). The initial and ending sections of the recording are clipped as they contain the effects of acceleration and reduction on the walking speed. Time is measured using a stopwatch, and then a person's gait velocity is computed (R Rana et al. 2016). Wearables, like FitBits and smart devices, have been used to predict gait speed, however, these devices are generally cloud-connected and can have issues with patient privacy where data leakage can occur at multiple points (Mills et al. 2016). GPS-based devices have also been used to track and monitor movement which can be used to analyse gait speed of a person, but these are not very accurate and can't work indoors (Bianchi et al. 2019). Given

the limitations of such devices, this study shows an alternative approach to using a microcontroller to develop an end-to-end solution to accurately analyse gait speed in a wide range of scenarios (outdoors, extended time, data privacy, etc.). In a paper (Das et al. 2020) the author has proposed automated techniques for an AI-enabled detection and classification of acute lymphocytic leukaemia. The author has used K-means clustering for lymphocyte extraction, features like shape, texture and colour are extracted through segmented images. Das et al. have also proposed (Das et al. 2022, 2022) the RBF kernel-based SVM for classification after reducing the dimension using PCA and achieved 96.05% accuracy, which is very high. Also presented in the paper, Grey-level co-occurrence matrix (GLCM) and grey-level run-length matrix (GLRLM) algorithms are used to extract the features of the nucleus.

Study	Main Contribution	Year	Performance	Recognition model
Studenski (Studenski et al. 2011)	Identified gait speed as a key indicator in survival analysis alongside traditional predictors, sparking increased research interest in the health implications of gait velocity	2011	Gait speed was calculated for each Participants used distance in meters and time in seconds.	Statistical Analysis -Kaplan-Meier product-limit survival curves
Sigg (Sigg et al. 2016)	Used Wi-Fi RSSI on smartphones to estimate walking speed, exploring non-wearable sensing modalities	2014	The performance of a passive RSSI-based system was evaluated, and the features mean, variance, signal peaks within 10% of the maximum, and the ratio between the first and second half of a feature window's means produced the best results.	Received signal strength indicator (RSSI) -based recognition
Hsu and Liu (Hsu et al. 2017b)	Developed an algorithm using radio signals to extract gait speed, introducing a device for in-home use	2017	The findings demonstrate that WiGait can record gait parameters even while a person moves freely and participates in a variety of home activities. It can also measure changes in gait velocity that are clinically relevant.	WiGait's algorithms are integrated with the WiTrack radio to deliver a stand-alone gait monitoring sensor.
Shi (Shi et al. 2019)	Explored wearable ankle robots for interpreting changes in walking speed in post-stroke rehabilitation	2019	Following a period of robot-assisted ankle rehabilitation training, all of the chosen clinical investigations demonstrated improvements in the dorsiflexion angle during the swing phase and propulsion on the paretic side during push-off.	Study provided a systematic investigation of existing wearable ankle rehabilitation robots.
Alhathi (Alharthi et al. 2019)	Reviewed continuous monitoring of human gait using deep learning for health conditions	2019	Multi-sensor and multi-modality fusion results in better accuracy and robustness.	Deep learning convolutional neural networks
Gupta, Anjali, et al. (Gupta et al. 2020).	Emphasized the use of physical therapists, clinicians and wearables for gait speed measurement, highlighted privacy concerns with cloud-connected devices	2020	The performance is also compared with SVM and KNN algorithms. The results showed that the classification accuracy of ELM is better than SVM and KNN.	artificial neural network is used to classify the human gait and compared with ELM.
R. Rana, D. et al. (R Rana et al. 2016)	Gait Velocity Estimation Using Time-Interleaved Between Consecutive Passive IR Sensor Activations	2016	The typical inaccuracy in estimating gait velocity is less than 2.5 cm/s. Compared to competing methodologies, our method is simple, economical and yields 20–100 times more gait velocity measurements per day.	support vector regression for gait velocity estimation
V. Bianchi et al. (Bianchi et al. 2019),	Discussed limitations of GPS-based devices for gait speed analysis, citing issues with accuracy and indoor use	2019	The system is conceived for daily activity monitor and nine different activities can be highlighted with an accuracy of 97%.	The sensor coupled to a convolutional neural network (CNN) network based low-cost or embedded devices.
P. K. Das, et al. (Das et al. 2020). P. K. Das, et al. (Das et al. 2022). P. K. Das, et al. (Das et al. 2022).	Introduced an end-to-end solution using a microcontroller for accurate gait speed analysis in various scenarios, addressing limitations of existing devices	2020, 2022	The quantitative examination of these strategies is done using a variety of performance measures, such as AUC, F1 score, J score, accuracy, precision, sensitivity and specificity.	An overview of the methods available for improving, segmenting, extracting features from and classifying the image with RBCs in order to identify sickle cell illness

3. Methodology

Our approach leverages the non-linearity advantages of deep neural networks to perform device classification of four gestures – running, walking, squatting and jumping. Specifically, we train and evaluate three different neural network models, based on the IMU sensor data collected from the microcontroller within this section we describe the details of our approach.

Figure 1 shows the detailed flow diagram of the proposed methodology. Table 1 shows different LED colour designed for different walking gesture.

3.1. Arduino Nano 33 BLE sense

The Arduino Nano 33 BLE sense (Figure 2) is a microcontroller, where the 'Nano' represents the board's tiny form factor having 45×18 mm dimensions, '33' outlines that board operating voltage at 3.3 V, 'BLE' is the capability of the board to receive and transmit data via Bluetooth Low Energy, and 'sense' indicates that the board has a multitude of environmental sensors built into it. Specifically, for our study, we utilised the 9-axis IMU (LSM9DSI chipset) on the board which contains accelerometer, gyroscope and magnetometer sensors. Finally, the main feature of this board is it is AI- an enabled ARM cortex M4 CPU (Nordic nRF52840 processor), which allows tensor computations and in turn gives us the possibility of running Edge Computing AI applications based on TinyML. The NINA B306 module allows for BLE protocol (operating at 2.4 GHz ISM band) and utilises a fraction of the power requirement of standard Bluetooth by only being active during communication. The trade-off here is the low data transmission rate, which is the case in our study where the requirement is to transmit only the sensor data during the collection stage and the predicted gesture during inferencing. Apart from the modules and sensors, the board also contains multi-colour LED lights, push buttons and a micro-USB connector.

3.2. Microcontroller software

The Arduino Nano board supports the execution of C code along with driver support for interfacing with onboard chipsets and sensors. The programming of the board can be done via the Arduino Software (IDE) which provides an easy-to-use multi-platform code development platform for the board. Within the interface we utilised Arduino developed libraries of Arduino LSM9DSI.h and Arduino BLE.h to utilise the LSM9DSI IMU sensor and the NINA B306 BLE module of the board. Two C programmes were developed, one for data capture and another for inferencing with the developed model. In the data capture programme, we sample the accelerometer, gyroscope and magnetometer readings of the IMU at 119 Hz or 119 sample/sec. A specific string format is created to encode the data for transmission via BLE. The board in this case acts as a BLE server where it listens for nearby clients (a local laptop in our case) to receive the data. Once the connection is established, the data is transmitted using the string format.

The inferencing programme is used when the model has been developed for the identification of gestures. The optimised and compressed model is converted into a binary header file (using the TensorFlow Lite compiler) and loaded up for inferencing on the board. Before inferencing, we allocate an area of memory (40 Kb) on the board for the computations needed for the deep-learned model. The programme leverages the TensorFlowLite.h to allow for the multiple optimised operations supported by the TFLite microcontroller library. The board then senses the IMU data at the same sampling rate of 119 Hz and passes the data through the neural model for inferencing. The programme then emits the results via BLE and LED actuation on the board.

3.3. Development environment

Apart from the Arduino IDE, the model development and board BLE interaction were done via TensorFlow and BLEAK software. Specifically, we leverage the Google TensorFlow library (along with Keras) for model training and evaluation along with TensorFlow Lite for the optimisation of operations for the microcontroller. All code is developed using Google Collaboratory, an interactive environment able to execute Python code as well as provide rich text and output results in the same file (notebook). We provide a link to the code repository (via Github) as a part of this paper, which contains the notebooks describing the data capture, preprocessing, model development, evaluation and model compression results.

For BLE interaction with the board, we leverage the Bluetooth Low Energy platform Agnostic Klient (BLEAK) software package. The package was used for discovering, connecting, reading, writing and getting notifications from the board. Specifically, we configured BLEAK on a laptop that acted as a client using a generic data protocol (GATT) to receive data from the board. The board in this case acted as a GATT server, where it transmitted the training data and inferencing results via BLE.

3.4. Model development

Once the data collection process is complete, the stored data is pre-processed to convert the raw samples to tensors and then the number of observations is split into training, test and validation datasets. Training and validation datasets are used in model development, where we train and evaluate three different models – 1) MLP, 2) CNN-LSTM (Khatun et al. 2022) and 3) CNN-GRU.

Based on the analysis of results (Khatun et al. 2023) (Precision, Recall and F1- score) and model size/complexity, we choose a single model for model compression. Within model compression, we prune and quantise the models to reduce size and complexity. At each stage of model, development, pruning and quantisation, we evaluate the performance using the test dataset. Figure 3 shows a high-level overview of our model development workflow. Below we describe each of the components in further detail:

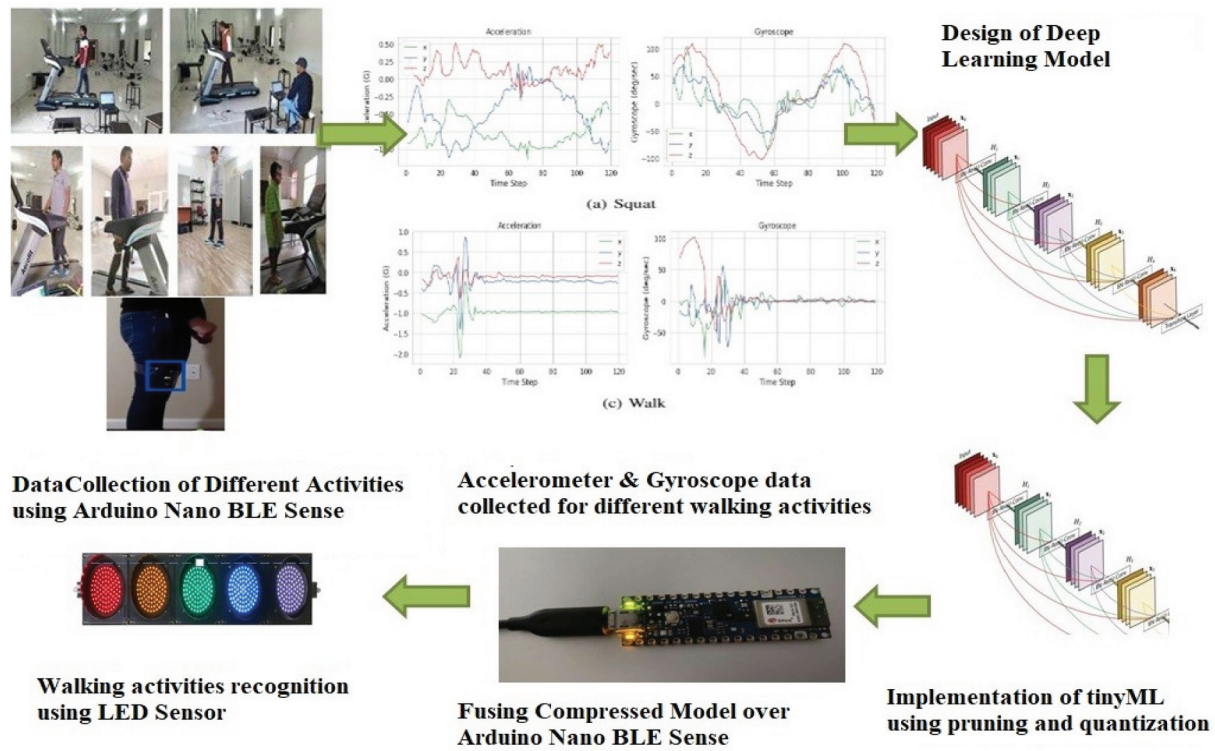


Figure 1. Flow diagram of proposed work.

Table 1. Gestures LED colour.

Gesture	LED Color
Squat	Red
Jump	Magenta
Walk	Blue
Run	Green
Other	White

- 1) *Data Collection & Preprocessing:* The data for five gestures – ‘Squat’, ‘Jump’, ‘Walk’, ‘Run’ and ‘Other’, is collected using the Arduino Board. The category ‘Other’ is used to classify

any other gesture that is not ‘Squat’, ‘Jump’, ‘Walk’, or ‘Run’, which generalises the models for the specific gesture categories. The board is programmed to capture the 3-axis accelerometer and gyroscope data using the inertial sensors (IMU).

The board acts as a central device sending the captured data via BLE to the execution environment of the BLEAK library that acts as a peripheral device using 16-bit String Characteristic UUID. The programmed board is then worn as a wearable just above the knee in a vertical orientation as shown in Figure 4 (see Figure 4). After establishing

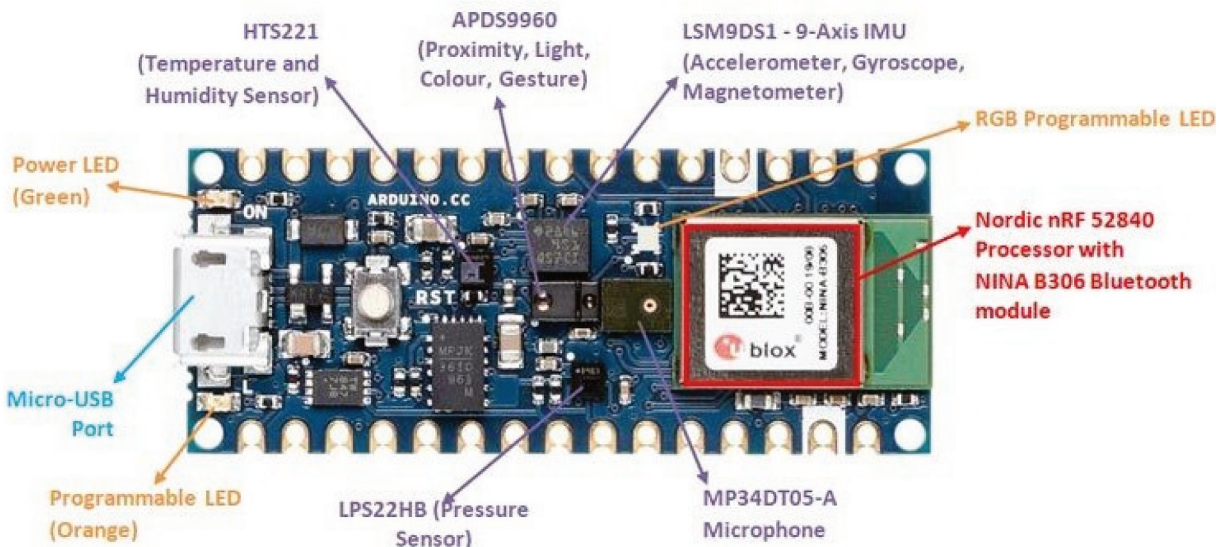


Figure 2. Arduino BLE with IMU sensor.

a connection with the peripheral, the board emits an orange LED, indicating it is ready for data transmission.

The data is then collected for each gesture performed 100 times with a sampling rate of 119 Hz. The board only captures the data if it has a minimum acceleration/gyroscope threshold of 1.5 (based on the metrics provided by the manufacturer). Each gesture approximately lasts for a second, ie we observe 119 accelerometer and gyroscope readings for each observation. Between each gesture, the participant pauses for 2 seconds before performing the next gesture. This enables data separation on the continuous observations transmitted by the board. The RESET button on the board or a cycle detection mechanism can also be used to achieve the same result.

Deep learning models accept input and generate output in the form of tensors. A tensor is a multi-dimensional array that can contain either numbers or other tensors. We transformed the raw input captured from the board into tensors, where a multi-dimensional tensor is created having a shape (500, 716). Five hundred indicates 100 samples for each of 5 gestures, and 714 represents the 3-accelerometer (aX, aY, aZ) and 3-gyroscope (gX, gY, gZ) data with a sampling rate of 119 per gesture that is $119 \times 6 = 714$. The input tensors are normalised between 0 and 1.

The output tensor has a shape (500 (Semwal and Nandi 2015), where 500 represents 100 samples for each of 5 gestures and 5 indicates the one-hot encoded matrix for each of 5 gestures.

The input and output data are shuffled into random order so that all the gesture data is evenly distributed into training, testing and validation sets. The shuffled data is split into 60% training, 20% test and 20% validation data.

3.4.1. Deep Learning Model Architecture

Here we describe the proposed three deep learning architectures MLP, CNN-LSTM and CNN-GRU for classification of gesture classification (Mutegeki et al. 2020). The first model of MLP takes three parameters optimiser, loss and metrics. The optimiser controls the learning rate. Within our study, we used the Adam optimiser with learning rate = 0.0001 was well suited in noisy learning environments with sparse gradients. We used the mean squared error as the loss function which is an average squared difference between the predicted and actual values,

where the value closer to 0 means the better the model performed. To calculate the accuracy of the model it is given as a parameter in metrics. The training/fit of the model was done over 700 epochs with a batch size of 10 observations in each epoch. To reduce the learning rate when a metric has stopped improving, ReduceLROnPlateau was used as a callback to monitor validation loss with a lower bound on the learning rate of 0.0001. This enables the model to reduce the learning rate by a factor of 10 each time the rate plateaus.

This is done to improve the overall accuracy of the model.

CNN-LSTM: The second model within our study aims to leverage the sequence-based data we observe from the IMU sensors. We explore the Recurrent Neural Networks (RNNs) based approach, which can be utilised to perform sequence-based prediction on the observed data. Specifically, we combine the efficacy of Convolutional Neural Networks (CNN) for pattern extraction with Long Short Term Memory (LSTM) to model the sequential data.

Convolution is the application of a sliding window filter to the input to extract high-level features from the raw data. The filter is applied repeatedly to the inputs which results in a map of activation that is known as called a feature map. This feature map indicates the location and the strength of a particular feature in the whole data. The max-Pool layer provides an approach for downsampling of feature map by presenting a summary of features (by selecting the max value) in patches of the feature map. A dropout rate of 50% is used after the Max-Pool layer to reduce overfitting. The model (Figure 5) uses 64 filters applied on each convolution layer (Muhammad et al. 2021).

The high-dimensional data is then flattened into a single-dimensional vector, which is then fed into LSTM units. Hochreiter et al. (Hochreiter and Schmidhuber 1997) first proposed the architecture of LSTM which was able to overcome the vanishing gradient limitations of the traditional RNN model by introducing the concepts of gates. The gates act as an information highway that can be used to retain long-term dependencies from earlier time steps. Within our model, we use 128 LSTM units to model the time sequence of feature maps extracted by the CNN filters.

Following the LSTM layer, the output is fed into two layers consisting of 1000 and 500 neurons in each. Neurons in these

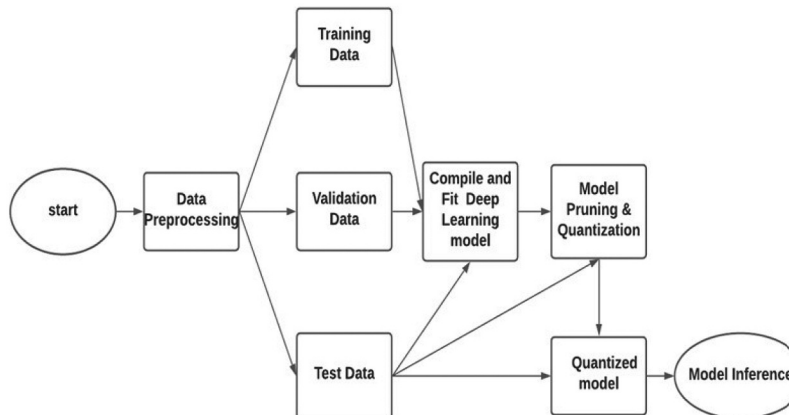


Figure 3. Deep learning workflow.

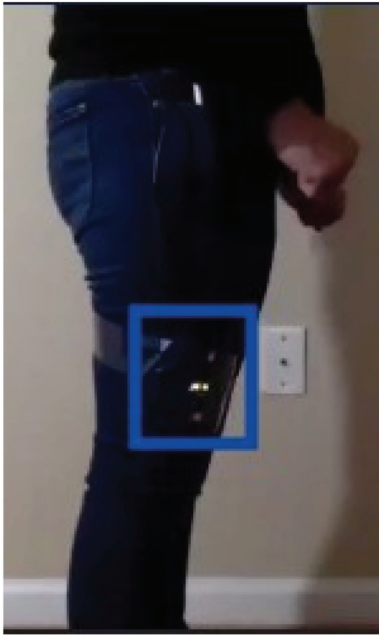


Figure 4. Arduino nano sense BLE on participant - the blue box shows the placement of the sensor on the participant's right thigh area.

layers are activated by ReLU. The layers are followed by a batch normalisation layer which aids in training speedup and accuracy. The final layer consists of five neurons (corresponding to the five gestures) activated by SoftMax.

Similar to the MLP model, we use ADAM as the optimiser and ReduceLROnPlateau to adjust the learning rate (starting at 0.001 and reducing to 0.0001). The model is trained for 50 epochs with 64 samples in each batch. Categorical cross-entropy is used as the loss function within the model.

CNN-GRU: Figure 6 shows the architecture of the third model used in the study. The architecture of the model is very similar to the LSTM-based architecture described in the aforementioned section. The model uses the same CNN layers to extract the feature maps from the data and reduce them using a Max-Pool layer. After flattening the output, the data is then fed into a Gated Recurrent Unit (GRU) layer.

GRU was first proposed by Cho et al. (Cho et al. 2014). Similar to LSTMs, the GRU architecture can model long-term dependencies in sequence-based data. However, GRUs are simpler and more efficient architecture than LSTMs, as they remove the cell state and use just the hidden state to transmit information across long distances. By removing the hidden cell state, the GRUs reduce the number of weights, activations and overall computations needed for the network.

Within the study, we use 128 GRU units within the sequence layer. The output of the layer is then fed into two dense feed-forward layers with 100 and 50 neurons each. The final layer consists of five neurons activated by SoftMax. Similar to the LSTM model, the GRU model uses Categorical cross-entropy as the loss function, ADAM as the optimiser and ReduceLROnPlateau, with the model trained for 50 epochs and 64 samples in each batch.

Each model is evaluated using 20% test data, where we record the model metrics of Precision, Recall and F1-score per gesture category.

3.4.2. Model Pruning and Quantization

A regular deep learning model is generally developed using a GPU and has a considerable size (in Mb) and large number of computations. For the model to work (inferencing) on an edge microcontroller, we are limited by the available space and the computational speed of the board. Within the Arduino Nano Sense board, we have about 1Mb of storage space available, out of which only about 600Kb is usable. For this reason, we had to reduce the model using compression techniques of pruning and quantisation.

Model compression (Han et al. 2019) techniques have been effectively used to develop deep learning architectures that are efficient both in space and compute complexity. One of the techniques is pruning, where redundant architecture modules are removed to make the network more compact. This can be done by gradually reducing the model weights to zero, enabling sparsity within the network and thereby decreasing the computational cost. Quantisation on the other hand, effectively lowers the precision of the weights and inputs to reduce the size of the network. Quantisation can be done during training where lower precision computations are performed to propagate the gradients or can be performed post-trained, where a mapping strategy can be used to convert the higher precision values to lower ones. Quantisation thus can effectively reduce the model size by utilising a lower bit storage for the weights of the model.

Within our study, we utilised magnitude-based weight pruning during the training of the model. Within the approach, after each epoch, the link (between neurons) with the smallest weight is zeroised (removed). The next epoch begins with the new weights and trains the model. In the training stage, we start with an initial sparsity of 65% and enable training of the model to reach 90% sparsity within the network. Specifically, we start with zeroing 65% of the weights and continue pruning till 90% of the weights are zero based on a polynomial decay function. The model is tested at each epoch with the validation dataset, and finally with the test data.

Following pruning, we perform post-training quantisation on the model. In the quantisation step, we perform linear mapping of the floating point weights of the model to the integer mapping of 8-bits, ie specific ranges of floating point values are converted into integers based on a linear map. This reduces the model size by 4× as we go from a storage resolution of 32 bits for floating point values to an 8-bit integer. For a full integer model with integer input and activations, we perform a calibration step by running inference on a small sample of training data (100 samples to map the input tensors, activations (intermediate layer outputs) and model output. This results in an end-to-end model with a full 8-bit integer specification of the model. Following the compression, we again test the models' accuracy using the test data.

3.4.3. Classification

Following the compression stage, we use the pruned and quantised model to programme the Arduino board for the classification of gestures. Tensor Flow Lite is used to convert the compressed model into a binary header file, which is then



Figure 5. Architecture of the CNN-LSTM network.

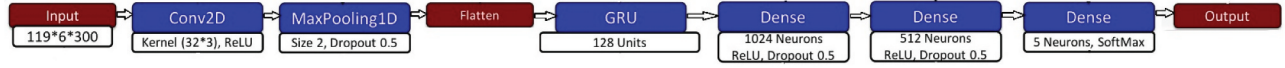


Figure 6. Architecture of the CNN-GRU network.

used for inference. We allocate 40 Kb of stack space on the board for the model to store and compute the results. The board is then initialised to receive the data, where it emits an orange LED light showing that it is receiving data from the IMU sensors.

If the input data crosses a minimum threshold of 1.5, the input of the accelerometer and gyroscope is fed into the model, where an inference step outputs the predicted probabilities of the data being in each gesture class (Squat, Run, Walk, Jump and Other). The highest probability gesture (along with its probability) is used to output the results via the LED board (LED colours Red, Green, Blue, Magenta and White corresponding to the gesture) and transmitted to the client device via BLE.

Algorithm 1 describes the overall workflow for the development and deployment of the gesture recognition model on the Arduino Nano Sense BLE board.

4. Results and discussion

Here we present the findings of our study. First, we discuss the results of the raw data analysis, followed by a performance of the three different deep learning models, and finally the results of the compressed model.

4.1. Gesture data

Figure 7 shows the raw accelerometer and gyroscope data plots for different types of gestures. Figure 7a shows the data for a single observation of squatting gestures. In both the accelerometer and gyroscope, we observe a cyclic trend in the data when the participant goes from a normal standing position to squatting and then gets back up to a normal position. In comparison, in the jump gesture (Figure 7b) the participant goes from a similar cyclic trend, ie jumping from a normal standing position and returning to standing, the x, y and z values show high fluctuations within the given window. This indicates a faster activity than squatting. The G values of acceleration and the degrees/sec for gyro readings are also substantially higher for jump than squat.

We observe the walking gesture in Figure 7c. Here, for standard walking speed (3 to 4 miles per hour), the graph shows that the frequency of a waveform-like structure is low, given the low number of steps taken in the fixed time interval. We also observe the accelerometer, x-axis and y-axis, follow similar cyclic patterns, concerning the time of different persons for the same joint type and walking speed. In comparison, the running gesture (Figure 7d) has higher accelerometer and

Algorithm 1: Human Activity Recognition using Compressed Deep Neural Networks (TinyML)

Result: Classification accuracy of human activity

initialization;

Step 1: Data Collection using IMU sensor, and represented in the form of (ax, ay, az, gx, gy, gz);

Step 2: Split the collected data set into three subsets i.e. train, test and validation sets;

Step 3: Develop deep learning models MLP, CNN-LSTM and CNN-GRU Neural Networks;

Step 4: Evaluate MLP, CNN-LSTM and CNN-GRU models using Recall, Precision and F1-score;

Step 5: Select a model based on accuracy, size and number of computations;

Step 6: Perform model compression (Pruning and Quantization) on the selected model;

Step 7: Validate the compressed model using the test data set;

Step 8: Load the model on an Arduino Nano board and perform inferencing;

Step 9: Output the results via LED indicator and BLE transmission to the client device;

gyroscope values, indicating a more vigorous activity (4 to 6 miles per hour). Also, within the same period, we observe more number of steps in run versus walk gestures. These can be identified by the peaks in the plots, where we observe two cycles present in the same one-second sample.

4.2. Deep learning model metrics

As described in the model development section (Section III-D), we evaluate the performance of three different deep learning model architectures – 1) Multi-Layer Perceptron (MLP), 2) CNN-Long Short Term Memory (CNN-LSTM) and CNN-Gated Recurrent Unit (CNN-GRU). Each of these models is evaluated with the 20% test data to record the metrics of precision, recall, f1-Score, and overall accuracy of the architectures. For each of the models we also observe the number of computations and the size of the resulting model. In Table 2, we show the metrics for each of the architectures. Overall, all models can perform with very high accuracy (greater than 88%) with the test data. Within the five gestures, squat and run are easier to identify for the models than the jump and walk gestures. For instance, considering the MLP model, we have almost perfect f1 and accuracy scores for squats and others, but for jump, walk and run gestures the model operates with an average weighted f1 score of about 0.88. The overall accuracy of the MLP model is 88%.

Comparing the simpler model to the CNN-LSTM and CNN-GRU models, we do observe an overall increase in accurately

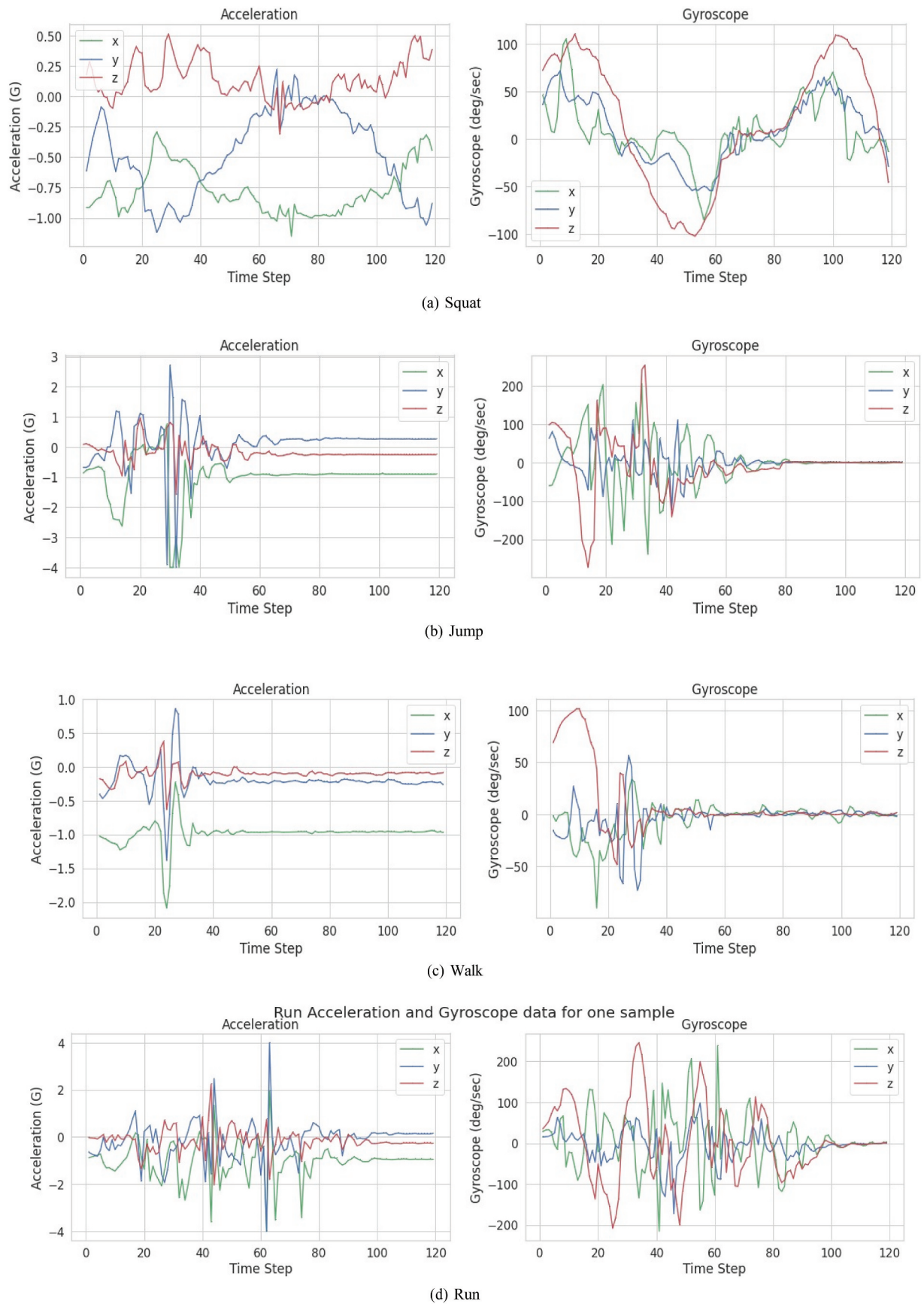


Figure 7. Accelerometer and gyroscope patterns for squat, jump, Walk and run for one sample.

identifying all five gestures. Similar to the MLP model, squatting and walking have higher f1-scores (0.90–0.95) than jumping, running and other (f1-score range 0.74–0.88). The overall accuracy of the CNN-LSTM model is 88%. The CNN-GRU model performs better than both the models with good f1-score in the squat, jump, walk and other (f1-score range 0.85–0.93), and only performs poorly in run activity with 0.74 f1-score. This leads to a higher accuracy of 89% overall.

Figure 8 shows the accuracy and the loss trends of the models at different epochs. For the MLP model, we initially evaluated the model for 700 epochs, and for the CNN-LSTM and CNN-GRU models, we evaluated them for 200 epochs each. Observing the graphs 8a, we concluded that the model training stabilises after around 450 epochs and we do not gain any additional accuracy after that. We also see the same in the loss analysis (Figure 8b) where the model's training and validation loss are close to each other at 450 epochs and start to diverge after that. Based on the observed results, we stopped the training of our model at 450 epochs for the MLP model. This prevents overfitting of the model. Within our study, we use the 'ReduceLROnPlateau' callback function, which reduces the learning rate when the validation loss metric has stopped improving (plateaued). This provides a gradual increase in training accuracy as observed in Figure 8a.

In the CNN-LSTM model (Figure 8(c,d)), we observe that the model's accuracy increases and the loss decreases to 50 epochs and then plateaus out. The training and validation curves in both the accuracy and loss metrics closely follow each other suggesting a very low chance of overfitting the model. In comparison, the CNN-GRU model (Figures 8(e,f)) does not

show any increase in model accuracy or drop in loss until 30 epochs. After approximately 30 epochs the model accuracy increases to 80 and then plateaus. Similarly in the loss plot 8f, the model continues reducing the loss until 80 epochs and then plateaus, thus 80 epochs can be considered as a good stopping point for model training.

Figure 9 shows the confusion matrices of the models when predicting with the test dataset. Here we observe that the MLP model (Figure 9(a)) is accurate on squat, jump and other gestures, where it can classify the gestures correctly in each class. In comparison, for the gesture run, it classified 22 observations, where it predicted five to be walking gestures and one jump and other gestures, respectively. As the patterns of jump motion can be similar to a walking or running gesture, this is where the model failed to make a correct prediction. The model also made incorrect predictions on the walking gesture, where it predicted 7 observations out of 24 to be of other gesture category.

Within the CNN-LSTM (Figure 9(b)) and CNN-GRU (Figure 9(c)), we observe that the models are mostly accurate in the squat, walk and jump categories. We do see some misclassifications by the CNN-LSTM model, where it incorrectly classified several run observations (4 out of 13) to be jump categories. This suggests for a high-velocity gesture such as running, the models are not able to accurately disambiguate the movement. It also misclassified 5 out of 19 observations of another category to be of the walk label. Similarly, in the CNN-GRU, it misclassified 3 out of 13 run observations to be walked and 4 out of 19 other observations to be walked. This goes to show that walking is one of the more difficult gestures for the models to correctly classify.

Table 2. Results of the deep learning models.

Model		Model Performance					Model Parameters		
		Squat	Jump	Walk	Run	Other	# Params	# Ops	Size (MB)
MLP	Accuracy	1.0	0.81	0.59	0.54	1.0	85,155	170,080	1.06 Mb
	Precision	1.0	1.0	1.0	0.59	0.86			
	Recall	1.0	0.85	0.68	1.0	1.0			
	F1	1.0	0.92	0.81	0.74	0.93			
CNN-LSTM	Accuracy	0.95	0.95	0.97	0.56	0.65	742,501	1,492,926	8.97 Mb
	Precision	0.95	0.84	0.95	0.90	0.93			
	Recall	0.95	0.95	0.96	0.69	0.74			
	F1	0.95	0.88	0.90	0.74	0.82			
CNN-GRU	Accuracy	0.95	1.0	0.93	0.70	0.65	722,277	1,459,646	8.73 Mb
	Precision	0.90	0.87	0.81	1.0	1.0			
	Recall	0.95	1.0	0.93	0.77	0.74			
	F1	0.93	0.93	0.87	0.74	0.85			
Previous Models (Hochreiter and Schmidhuber 1997; Dua et al. 2023, 2023)	Accuracy	0.92	0.98	0.91	0.65	0.62	922,477	1,659,646	10.73 Mb
	Precision	0.87	0.84	0.80	0.99	0.97			
	Recall	0.91	0.96	0.89	0.78	0.74			
	F1	0.94	0.91	0.88	0.71	0.81			

Table 3. Results of the MLP compressed Model.

Model		Model Performance					Model Size
		Squat	Jump	Walk	Run	Other	
Pruned	Accuracy	1.0	0.83	0.88	1.0	1.0	342.7 Kb
	Precision	1.0	1.0	0.96	0.87	0.86	
	Recall	1.0	0.85	0.89	1.0	1.0	
	F1	1.0	0.92	0.93	0.93	0.93	
Quantized	Accuracy	1.0	0.83	0.97	1.0	0.95	88.8 Kb
	Precision	1.0	1.0	0.93	0.87	0.95	
	Recall	1.0	0.85	0.96	1.0	0.95	
	F1	1.0	0.92	0.95	0.93	0.95	

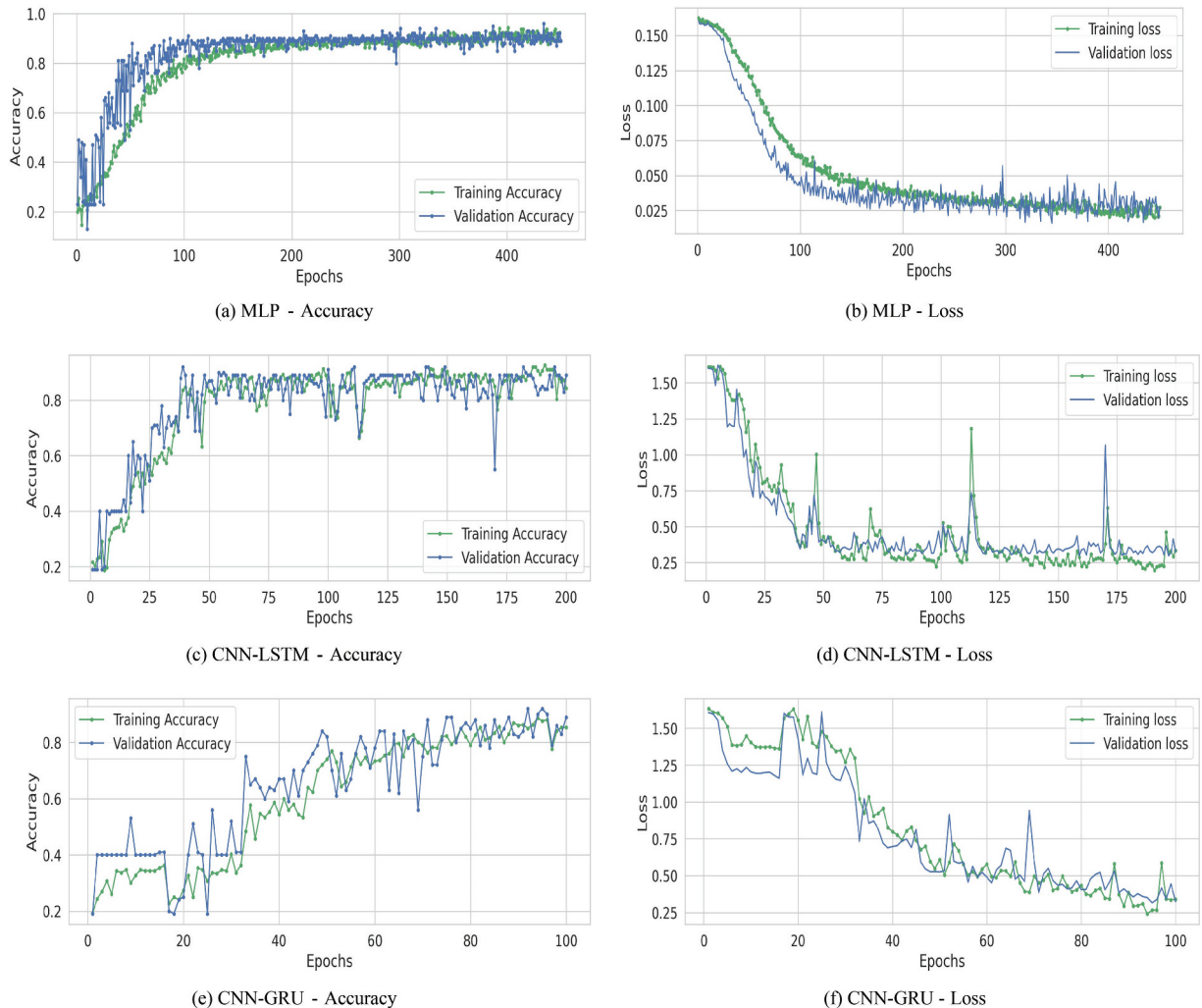
Table 4. Results of the CNN-LSTM compressed Model.

Model		Model Performance					Model Size
		Squat	Jump	Walk	Run	Other	
Pruned	Accuracy	0.99	0.79	0.78	0.99	0.94	1907.2 Kb
	Precision	0.99	1.0	0.94	0.84	0.84	
	Recall	0.99	0.83	0.84	0.98	0.97	
	F1	0.99	0.92	0.93	0.93	0.93	
Quantized	Accuracy	0.99	0.82	0.97	1.0	0.95	888.8 Kb
	Precision	0.99	0.99	0.95	0.84	0.92	
	Recall	0.99	0.82	0.95	1.0	0.95	
	F1	0.99	0.90	0.93	0.93	0.95	

While we do not present the results here our initial models did not consist of the other class. In this scenario, the model accuracy was below 80%, and overfitting to the data was occurring. By including samples of random gestures, we were able to improve the generalisation of the model and in turn the accuracy (reported in the results). We also evaluated multiple optimisers and observed Adam to provide the best model accuracy due to its capability of handling sparse gradients (partly due to the low sample size used in the study).

Table 2 also shows the number of parameters and the model size for each of the developed models. Here we observe that the MLP model has the lowest number of parameters (85,155)

and floating point operations (170,080) and correspondingly has the lowest size at 1.06 Mb. In comparison, the CNN-LSTM and CNN-GRU models have almost 9 times the number of parameters (742,501–722,277) and operations (American Psychological Association 2010) 492,926 - (American Psychological Association 2010) 459,646, leading a very large file size of the model (8.97–8.73 Mb respectively). While the CNN-GRU does show a 1% overall increase in accuracy over the MLP model, the size of the model is a key issue in deploying it on the microcontroller (available 600 Kb). For this reason, we proceeded with the compression of the MLP model only and deployed it on the microcontroller.

**Figure 8.** Model accuracy and loss plots for MLP, CNN-LSTM and CNN-GRU using test data.

4.3. Model compression results

In Table 5 we show the results of the model's performance after pruning and quantising the MLP model. As described in earlier sections, we use a 90% sparsity level for pruning and an 8-bit integer representation for quantisation. In the pruning stage, we observe an increase in prediction performance across the categories of jump, walk, run and other, in comparison to the baseline MLP model. The overall average F1 score increases from 88% to 94%. Figure 10a shows the confusion matrices for the predictions made by the pruned model. Here we observe the model is making fewer mistakes with the jump, walk and run categories when compared to the baseline MLP model (Figure 9a). The increase in performance can be attributed to the better generalisability of the pruned architecture, where the sparsity of the network enables learning of the higher-level feature representations of the gestures.

The results for the quantisation show that the compressed quantised model is even improving on the pruned model. We observe an increase in the prediction accuracy of the walking gesture, where the model has an average F1-score of 0.95 (compared to 0.91 for pruned). The same is shown in the confusion matrix (Figure 10b), where the model only makes an error on a single observation out of 28 walking gestures. This leads to an overall average F1 score of 95%.

Within the dual stages of model compression, we see an increase from 7% (88% MLP baseline model to 95% quantised model). The higher accuracy of the quantised model can also be attributed to the increase in generalisability with lower resolution of the data used for model weights (Int8 vs. Float32). This enables the model to ignore the smaller fluctuations in the data and learn the higher-level patterns within the gestures.

Tables 3, 4 and 5 shows the resulting file sizes of the models. With pruning, we observe a 3× compression in file size at 342Kb. The resulting file can be used as it stands with the microcontroller, as it would fit the cache space. With quantisation, we observe another 4× decrease in file size with the lower resolution Int8 being used for storage of the model. The quantised file is then converted to a c header file, consisting of only the binary weights and activations using the TensorFlow lit for the micro compiler.

4.4. Deployment on arduino nano sense BLE

The compressed model header file along with the Arduino sketch file (.ino file) is loaded up on the board via USB. The board is then reset to activate the new sketch and a battery is

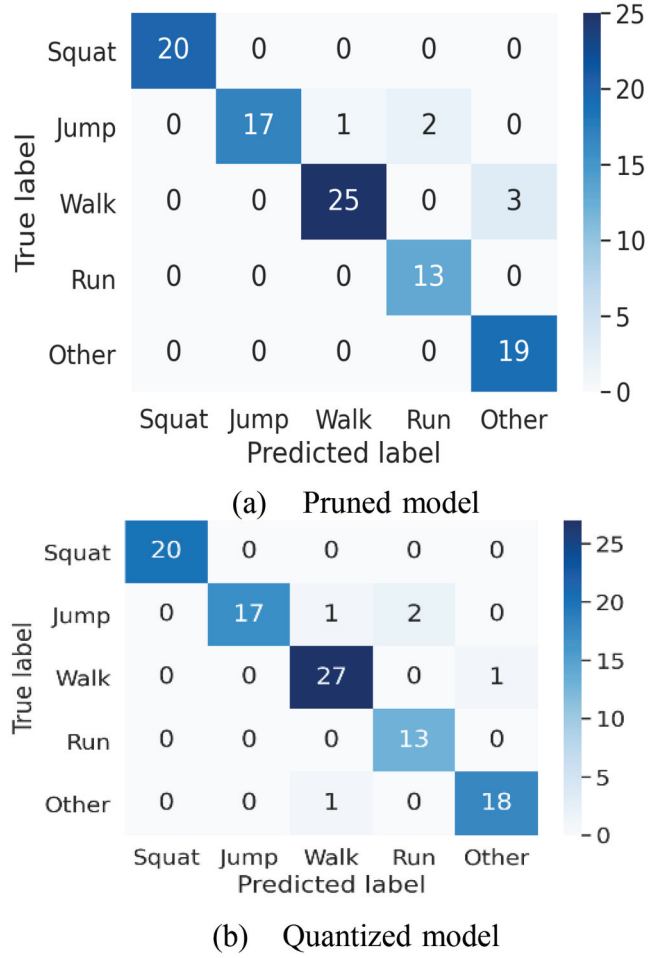


Figure 10. Confusion matrix for compressed MLP model.

connected to it. Following this, we ask the participant to perform the different gestures and the board is allowed to infer in real time.

Figure 11 shows the outputs of the board, which lights up the LED with the corresponding colour of the gesture (Table 1). At the same time, it also sends out a string message over BLE (to the data collection computer) with the gesture category (and the probability value) it classified during its inferencing. We evaluated the performance of the board across 100 gestures (20 for each category) on the participant. The prediction performance of the model deployed on the board is very similar to the results observed by the quantised model. Across all of the

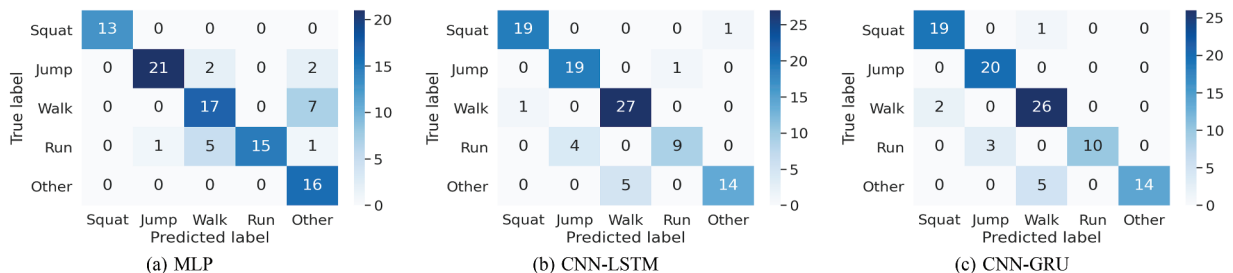
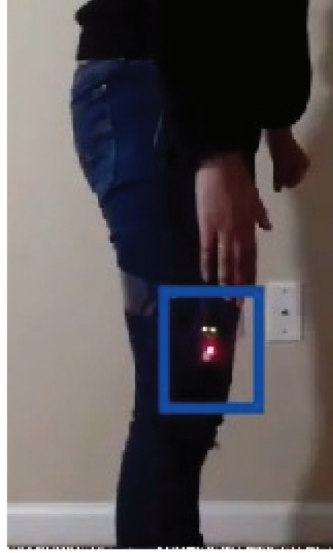


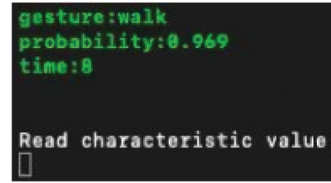
Figure 9. Confusion matrices for MLP, CNN-LSTM and CNN-GRU models showing the classification of the gestures with the test data.

Table 5. Results of the CNN-GRU compressed model.

Model	Model Performance						Model Size
		Squat	Jump	Walk	Run	Other	
Pruned	Accuracy	0.98	0.82	0.87	0.99	0.99	1200.7 Kb
	Precision	0.96	1.0	0.93	0.87	0.86	
	Recall	0.98	0.82	0.89	0.99	0.99	
	F1	0.98	0.91	0.93	0.93	0.93	
Quantized	Accuracy	0.99	0.83	0.97	1.0	0.95	600.6 Kb
	Precision	0.99	1.0	0.91	0.83	0.94	
	Recall	0.99	0.85	0.96	0.99	0.93	
	F1	0.99	0.91	0.93	0.92	0.92	



(a) Arduino LED Prediction



(b) Arduino BLE Prediction

Figure 11. On device inferencing - predictions made by Arduino Nano sense BLE on the participant. The LED lights up with the color indicating the gesture and the gesture class (and its probability) is transmitted over bluetooth low energy.

gestures, we observed 100–500 milli second inference time for most of the activities. This inference time can further be reduced by removing the BLE transmission of the results.

5. Conclusion

This research work presented human activity/gesture recognition using a compressed deep learning model. A total of five gestures are considered for the experiment jump, run, walk, squat and other. We have achieved 88%, 88.1% and 89% classification results for three models of deep learning namely, MLP, CNN-LSTM and CNN-GRU, respectively. Using model compression techniques of pruning and quantisation, we were able to increase the performance of the models to over 95% while resulting in a 12× model size reduction. An Arduino Nano 33 BLE microcontroller was used for data collection with an in-build IMU. The microcontroller requires very little energy for processing power, memory and storage due to its tiny size. It is easy to wear, ‘flexible’ to use, and most importantly maintains

‘privacy’ of data (with on-device computing). Apart from this the board also has IMU sensor capabilities, Bluetooth low energy connectivity, and the capacity to run deep learning network models using ARM cortex M processors. Within the study, we deployed the compressed model on the board and performed real-time inferencing of gestures with it. The results show a promising new avenue in human activity recognition with low-powered, secure and accurate edge AI devices.

Disclosure statement

No potential conflict of interest was reported by the author(s)

Funding

This work is supported by the DST, Govt. of India, and Research under the SERB [Empowerment and Equity Opportunities for Excellence in Science], Project EEQ/2021/000502. The publicly available data set can be downloaded for research purposes: Data-set Link.

Notes on contributors



National Institute of Technology Jamshedpur, Jharkhand, India. Her current research interests are IoT, IoMT, and Machine Learning.



Amulya Yadagani received her Bachelors in Software Engineering from GITAM University (India) and her Masters in Computer Science from University of North Carolina - Greensboro. Her research interests include Data Science, Machine Learning, and Embedded Programming.



Basudeba Behera received the Bachelor of Engineering degree in Electronics and Instrumentation Engineering and the Master of Technology degree in Electronics and Communication Engineering from GIET University, Odisha, India, in 2006 and 2009, respectively. He completed his Ph.D. degree in the Department of Electronics and Electrical Engineering from Indian Institute of Technology Guwahati (IITG) in Assam, India, in 2017.

Currently, he is working as a full-time faculty member (Assistant Professor) at National Institute of Technology Jamshedpur (NITJSR). He has been involved in research on Piezoelectric and SAW devices, Ultrasonic Motors, mostly in Modelling, simulation, and fabrication of the SAW motors.



Vijay Bhaskar Semwal currently working as Assistant Professor in Department of CSE, MANIT Bhopal. He has also served as an Assistant Professor in the Department of CSE at NIT Rourkela, IIIT Dharwad and NIT Jamshedpur. Currently, he is supervising one DST sponsored project on human gait pattern generation for humanoid robots walk Simulation. His research interests are machine learning, deep learning, human activities recognition, analysis of biped locomotion and humanoid push recovery.



Somya Mohanty is currently working as an Assistant Professor in the Department of Computer Science at the University of North Carolina –Greensboro. Prior to this, Dr. Mohanty worked as Assistant Research Professor (and lead the Innovative Data Laboratory) at Social Science Research Center – Mississippi State University. He received his B.S. in Computer Science and Engineering from Berhampur University, M.S. in Computer Science

from Florida State University, and Ph.D. degree in Computer Science and Engineering from Mississippi State University. Research interests include big-data, machine learning, data science, cyber-security, and trustworthy computing, and specializing in large-scale databases, data-mining, algorithm design, web application, network security, trustworthy platform development, and computer

References

- Alharthi AS, Yunas SU, Ozanyan KB. 2019. Deep learning for monitoring of human gait: a review. *IEEE Sensors*. 19(21):9575–9591. 1 Nov.1. doi: [10.1109/JSEN.2019.2928777](https://doi.org/10.1109/JSEN.2019.2928777).
- American Psychological Association. 2010. Publication manual of the American psychological association. 6th ed. Washington DC: Canadian Psychological Association; p. 272.
- Banbury CR, et al. Benchmarking TinyML systems: challenges and direction. arXiv preprint arXiv:2003.04821, 2020.
- Bartolo D. 2020. Sensorized assessment of dynamic locomotor imagery in people with stroke and healthy subjects. *Sensors*. 20(16):4545. doi:[10.3390/s20164545](https://doi.org/10.3390/s20164545).
- Behera B. 2021. “Regular self-health monitoring and medicine reminder system with emergency alert messaging using IoT”, book title “internet of things and its applications, ISSN-1876-1100. Springer. doi: [10.1007/978-981-16-7637-6](https://doi.org/10.1007/978-981-16-7637-6).
- Bianchi V, Bassoli M, Lombardo G, Fornacciari P, Mordonini M, De Munari I. 2019Oct. IoT wearable sensor and deep learning: an integrated approach for personalized human activity recognition in a smart home environment. *IEEE Internet Things J*. 6(5):8553–8562. [10.1109/JIOT.2019.2920283](https://doi.org/10.1109/JIOT.2019.2920283).
- Bijalwan V, Semwal VB, Mandal TK. 2021. Fusion of multi-sensor-based biomechanical gait analysis using vision and wearable sensor. *IEEE Sensors*. 21(13):14213–14220. 1 July1. doi: [10.1109/JSEN.2021.3066473](https://doi.org/10.1109/JSEN.2021.3066473).
- Brull A, Zubizarreta A, Cabanes I, Rodriguez-Larrad A. 2020. “Sensorized tip for monitoring people with multiple sclerosis that require assistive devices for walking. *Sens (Basel)*. 20(15):4329. Aug 3. doi: [10.3390/s20154329](https://doi.org/10.3390/s20154329).
- Chen K. 2021. Deep learning for sensor-based human activity recognition: overview. *Chall Oppo*. 37. reprint. 2001.07416, arXiv.
- Cho K, van Merriënboer B, Gulcehre C, Bahdanau D, Bougares F, Schwenk H, Bengio Y. 2014. Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation. arxiv. doi: [10.48550/arXiv.1406.1078](https://doi.org/10.48550/arXiv.1406.1078).
- Das PK, A DV, Meher S, Panda R, Abraham A. 2022. A systematic review on recent advancements in deep and machine learning based detection and classification of acute lymphoblastic leukemia. *IEEE Acces*. 10:81741–81763. doi: [10.1109/ACCESS.2022.3196037](https://doi.org/10.1109/ACCESS.2022.3196037).
- Das PK, Meher S, Panda R, Abraham A. 2020. A review of automated methods for the detection of sickle cell disease. *IEEE Rev Biomed Eng*. 13:309–324. doi: [10.1109/RBME.2019.2917780](https://doi.org/10.1109/RBME.2019.2917780).
- Das PK, Meher S, Panda R, Abraham A. 2022Oct. An efficient blood-cell segmentation for the detection of hematological disorders. *IEEE Trans Cybern*. 52(10):10615–10626. [10.1109/TCYB.2021.3062152](https://doi.org/10.1109/TCYB.2021.3062152).
- Das PK, Sahoo B, Meher S. 2022. An efficient detection and classification of acute leukemia using transfer learning and orthogonal softmax Layer-Based Model. *IEEE/ACM Trans Comput Biol Bioinformatics*. 20(3):1817–1828. doi: [10.1109/TCBB.2022.3218590](https://doi.org/10.1109/TCBB.2022.3218590).
- De Capua C. 2009. A remote doctor for homecare and medical diagnoses on cardiac patients by an adaptive ECG analysis. 2009 IEEE International Workshop on Medical Measurements and Applications; Cetraro, Italy. IEEE.
- De Leonardis G. 2018. Human activity recognition by wearable sensors: comparison of different classifiers for real-time applications. *IEEE International Symposium on Medical Measurements and Applications (MeMeA)*; Rome, Italy. IEEE.
- Dua N. 2023. A survey on human activity recognition using deep learning techniques and wearable sensor data. *Machine Learning, Image Processing, Network Security and Data Sciences: 4th International Conference, MIND 2022, Virtual Event, January 19–20, Cham: Springer*.
- Dua N, Singh SN, Semwal VB, Challa SK. 2023. Inception inspired CNN-GRU hybrid network for human activity recognition. *Multimedia Tools Appl*. 82(4):5369–5403. doi: [10.1007/s11042-021-11885-x](https://doi.org/10.1007/s11042-021-11885-x).
- Gravina R, Ma C, Pace P, Aloï G, Russo W, Li W, Fortino G. 2017. Cloud-based activity-aaS cyber-physical framework for human activity monitoring in mobility. *Future Gener Comput Syst*. 75:158–171. doi: [10.1016/j.future.2016.09.006](https://doi.org/10.1016/j.future.2016.09.006).
- Gupta A, Semwal VB. 2020. Multiple Task Human Gait Analysis and Identification: Ensemble Learning Approach. In: Mohanty SN, editor. *Emotion and Information Processing*. Cham: Springer. doi: [10.1007/978-3-030-48849-9_12](https://doi.org/10.1007/978-3-030-48849-9_12).
- Han YC, Wong KI, Murray I. 2019. Gait phase detection for normal and abnormal gait using IMU. *IEEE Sensors*. 19(9):3439–3448. 1 May1. doi: [10.1109/JSEN.2019.2894143](https://doi.org/10.1109/JSEN.2019.2894143).

- Hochreiter S, Schmidhuber J. 1997. Long short-term memory. *Neural Comput.* 9(8):1735–1780. doi: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- Hsu C-Y. 2017a. Extracting gait velocity and stride length from surrounding radio signals. *Conference on Human Factors in Computing Systems*, ACM; Denver, CO, USA. p. 2116–2126.
- Hsu C-Y. 2017b. Extracting gait velocity and stride length from surrounding radio signals. *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*; Denver, CO, USA.
- Jain R, Semwal VB, Kaushik P. 2022. Stride segmentation of inertial sensor data using statistical methods for different walking activities. *Robotica*. 40(8):2567–2580. doi: [10.1017/S026357472100179X](https://doi.org/10.1017/S026357472100179X).
- Khatun MA. 2023. Deep CNN-GRU based human activity recognition with automatic feature extraction using smartphone and wearable sensors (ECCE- 2023). Bangladesh. [10.1109/ECCE57851.2023.10101550](https://doi.org/10.1109/ECCE57851.2023.10101550).
- Khatun MA, Yousuf MA, Ahmed S, Uddin MZ, Alyami SA, Al-Ashhab S, Akhdar HF, Khan A, Azad A, Moni MA. 2022. Deep CNN-LSTM with self-attention model for human activity recognition using wearable sensor. *IEEE J Transl Eng Health Med*. 10:1–16. Art no. 2700316. doi: [10.1109/JTEHM.2022.3177710](https://doi.org/10.1109/JTEHM.2022.3177710).
- Mills AJ, Watson RT, Pitt L, Kietzmann J. 2016. Wearing safe: physical and informational security in the age of the wearable device. *Elsevier- Bus Horizons*. 59(6):615–622. doi: [10.1016/j.bushor.2016.08.003](https://doi.org/10.1016/j.bushor.2016.08.003).
- Mim TR, Amatullah M, Afreen S, Yousuf MA, Uddin S, Alyami SA, Hasan KF, Moni MA. 2023. GRU-INC: an inception-attention based approach using GRU for human activity recognition. *Expert Syst Appl*. 216:119419. ISSN 0957-4174. doi: [10.1016/j.eswa.2022.119419](https://doi.org/10.1016/j.eswa.2022.119419).
- Muhammad K M, Ullah A, Imran AS, Sajjad M, Kiran MS, Sannino G, de Albuquerque VHC. 2021. Human action recognition using attention based LSTM network with dilated CNN features. *Future Gener Comput Syst*. 125:820–830. ISSN 0167-739X. doi: [10.1016/j.future.2021.06.045](https://doi.org/10.1016/j.future.2021.06.045).
- Mutegeki R. 2020. A CNN-LSTM approach to human activity recognition. *2020 International Conference on Artificial Intelligence in Information and Communication (ICAIC)*, Fukuoka, Japan. pp. 362–366. doi: [10.1109/ICAIC48513.2020.9065078](https://doi.org/10.1109/ICAIC48513.2020.9065078).
- R Rana R, Austin D, Jacobs PG, Karunanithi M, Kaye J. 2016. Gait velocity estimation using time-interleaved between consecutive passive IR sensor activations. *IEEE Sensors*. 16(16):6351–6358. Aug.15. doi: [10.1109/JSEN.2016.2577708](https://doi.org/10.1109/JSEN.2016.2577708).
- Semwal VB, Gaud N, Lalwani P, Bijalwan V, Alok AK. 2022aFeb. Pattern identification of different human joints for different human walking styles using inertial measurement unit (IMU) sensor. *Artif Intell Rev*. 55 (2):1149–1169. [10.1007/s10462-021-09979-x](https://doi.org/10.1007/s10462-021-09979-x).
- Semwal VB, Gaud N, Lalwani P, Bijalwan V, Alok AK. 2022b. “Pattern identification of different human joints for different human walking styles using inertial measurement unit (IMU) sensor. *Artif Intell Rev*. 55 (2):1149–1169. doi: [10.1007/s10462-021-09979-x](https://doi.org/10.1007/s10462-021-09979-x).
- Semwal VB, Kumar C, Mishra PK, Nandi GC. 2018Jan. Design of vector field for different subphases of gait and regeneration of gait pattern. *IEEE Trans Automat Sci Eng*. 15(1):104–110. [10.1109/TASE.2016.2594191](https://doi.org/10.1109/TASE.2016.2594191).
- Semwal VB, Nandi GC. 2015April. Toward developing a computational model for bipedal push recovery—A brief. *IEEE Sensors*. 15 (4):2021–2022. [10.1109/JSEN.2015.2389525](https://doi.org/10.1109/JSEN.2015.2389525).
- Shi B, Chen X, Yue Z, Yin S, Weng Q, Zhang X, Wang J, Wen W. 2019. Wearable ankle robots in post-stroke rehabilitation of gait: a systematic review. *Front Neurobot*. 13:63. doi: [10.3389/fnbot.2019.00063](https://doi.org/10.3389/fnbot.2019.00063).
- Sigg S. 2016. The telepathic phone: frictionless activity recognition from wifi-rssi. *2014 IEEE international conference on pervasive computing and communications (PerCom)*; Budapest, Hungary. IEEE.
- Studenski S, et al. 2011. Gait speed and survival in older adults. *JAMA*. 305 (1):50–58. doi:[10.1001/jama.2010.1923](https://doi.org/10.1001/jama.2010.1923).
- Vrigkas M, Nikou C, Kakadiaris IA. 2015. A review of human activity recognition methods. *Front Robot AI*. 2:28. doi: [10.3389/frobt.2015.00028](https://doi.org/10.3389/frobt.2015.00028).
- Zebin T. 2015. Inertial sensing for gait analysis and the scope for sensor fusion. *2015 IEEE SENSORS*. Busan, Korea (South). pp. 1–4. doi: [10.1109/ICSENS.2015.7370262](https://doi.org/10.1109/ICSENS.2015.7370262).